Siew Hean — Project Portfolio Page

Overview

JobSeek is a command-line desktop application (CLI) designed for undergraduate students and job seekers to track and manage their internship and graduate-role applications, monitor upcoming deadlines, and organize their recruitment pipeline. Optimized for keyboard-driven efficiency, it allows applicants to record opportunities in seconds, track progress through recruitment stages (`Applied`, `Online Assessment`, `Interview`, `Offer`, `Accepted`, `Rejected`, `Withdrawn`), filter active vs. completed applications, and search and sort entries seamlessly.

Summary of Contributions

- **Code contributed**: tP Code Dashboard (RepoSense)

Enhancements implemented

- **Interactive Numbered Menu Architecture & CLI Workflow Lifecycle** (`Main.java`, `Ui.java`) (#1, #63, #74):
  - *What it does*: Architected the end-to-end numbered menu system (`[0]` to `[10]`) in `Main.java` and `Ui.java`, managing application lifecycle transitions with structured validation, clean spacing, and automatic menu redisplay after task execution.
  - *Justification & Technical Depth*: Decoupled menu command dispatch from I/O loops, reordered menu numbering so `[10] Help` serves as the final action option, suppressed console log pollution from `java.util.logging`, and established graceful EOF termination and reprompt loops for invalid entries.
- **Edit Application Engine & Structured Dual-Column Field Menu** (`EditCommand`, `Ui.java`) (#24, #74):
  - *What it does*: Engineered in-place application field modification allowing users to modify specific fields (`[1]` Company to `[8]` All) in place without deleting and recreating records, including clearing optional fields via blank inputs.
  - *Justification & Technical Depth*: Upgraded the cluttered single-line prompt into a structured, dual-column edit menu grid (`Ui.showEditMenu()`) matching JobSeek's interface style. Integrated full state-delta tracking across all 8 fields for multi-level `undo` restoration.
- **Comprehensive 9-Column ASCII Table View** (`ListCommand`) (#86):
  - *What it does*: Expanded `ListCommand` from a 5-column table into a comprehensive 9-column ASCII table showing all 8 application fields (`No.`, `Completed`, `Company`, `Role`, `Stage`, `Deadline`, `Portal Link`, `Email`, `Notes`).
  - *Justification & Technical Depth*: Dynamically calculates column widths across all active and completed records based on content lengths to guarantee alignment, and seamlessly formats empty optional fields with `-`.
- **Recruitment Pipeline Stage Engine & Flexible Normalization** (`RecruitmentStageCommand`, `RecruitmentStage`) (#33):
  - *What it does*: Engineered recruitment stage tracking (`Applied`, `Online Assessment`, `Interview`, `Offer`, `Accepted`, `Rejected`, `Withdrawn`) with terminal stage indicators.
  - *Justification & Technical Depth*: Built `fromChoice()` and `fromLabel()` supporting numbers `1`–`7`, case-insensitive names, and shortcuts (`oa` for Online Assessment, `pending` for Applied), decoupled into dedicated command handlers.
- **Extended Domain Attributes: Portal Link, Contact Email & Interview Notes** (#56):
  - *What it does*: Extended domain entities, command payloads, and `Storage` to track company portal URLs, recruiter emails, and interview notes.
  - *Justification & Technical Depth*: Engineered backward compatibility in `Storage` to seamlessly ingest legacy 4-column and 5-column save files while serializing updated 8-column records.
- **Quality Assurance Leadership, Equivalence Partitioning & Boundary Testing** (#62):
  - *What it does*: Configured JaCoCo test reporting and authored `TestCaseDesignTest.java` implementing formal Equivalence Partitioning (EP) and Boundary Value Analysis (BVA).
  - *Justification & Technical Depth*: Raised repository test suite to **over 315 automated tests** with **100% class coverage** and **97.5% line coverage**, stress-testing boundary offsets (D-1, D, D+7, D+8), integer overflows, and corrupted storage schemas.

Contributions to the User Guide (UG)

- Authored core user guide sections: Adding applications, Listing applications (all 9 columns), Editing applications (with the structured dual-column field menu), Recruitment stage tracking (numeric options and aliases), and Help / Command Summary quick reference tables.
- Structured the user guide into an intuitive step-by-step user journey: **Add → List → View → Edit → Status → Search → Help**.

Contributions to the Developer Guide (DG)

- **Architecture Overview & Standardization**: Authored Architecture Overview and Architecture Diagram (User, UI, Logic, Model, Storage). Standardized and regenerated all UML diagrams across the DG using Mermaid.

- **Class & Sequence Diagrams**: Authored Overall Architecture Class Diagram, Sequence Diagrams for Add, Edit, List, Search, Sort, Storage, and Architecture.
- **Design Sections**: Authored design sections for Edit Application & Recruitment Stage tracking, Help Command, and Automated Testing / Code Coverage.
- **Manual Testing Instructions**: Authored end-to-end Manual Testing instructions with exact console representations for add, list, edit, status, and help.

Contributions to team-based tasks

- **Release Management**: Packaged, validated, and published binary releases for Milestone 1 (`v1.0.0`) and Milestone 2 (`v2.0.0`) with release notes and executable JAR binaries.
- **Milestone & Project Management**: Created and managed GitHub Milestone `v2.1` and maintained the issue tracker, prioritizing deliverables and triaging bugs.
- **Quality Assurance Leadership**: Enforced strict Java 25 and Checkstyle compliance across PRs, maintaining a zero-warning build standard.
- **Product Documentation**: Overhauled project `README.md` and co-authored `docs/AboutUs.md`.

Review/mentoring contributions

- Actively reviewed teammates' pull requests (e.g. PR #16, #24, #44, #45, #48, #51, #52, #54), catching regression risks and design flaws.
- Resolved complex merge conflicts across shared modules (`Main.java`, `Application.java`, `Storage.java`, `UserGuide.md`, `DeveloperGuide.md`).
- Mentored teammates on Checkstyle rule resolution, Gradle build tasks, and Java 25 compatibility.

Contributions beyond the project team

- Conducted systematic quality audits and bug reporting on other teams' products during the Practical Exam Dry Run (PE-D), submitting detailed reproduction steps, severity assessments, and suggestions.
- Actively contributed to class-wide peer reviews, providing technical feedback on architecture documentation and testing methodologies.